

Chapter outline

Section 30.1 presents two ways to represent polynomials: the coefficient representation and the point-value representation. The straightforward method for multiplying polynomials of degree n —equations (30.1) and (30.2)—takes $\Theta(n^2)$ time with polynomials represented in coefficient form, but only $\Theta(n)$ time with point-value form. Converting between the two representations, however, reduces the time to multiply polynomials to just $\Theta(n \lg n)$. To see why this approach works, you must first understand complex roots of unity, which Section 30.2 covers. Section 30.2 then uses the FFT and its inverse to perform the conversions. Because the FFT is used so often in signal processing, it is often implemented as a circuit in hardware, and Section 30.3 illustrates the structure of such circuits.

This chapter relies on complex numbers, and within this chapter the symbol i denotes $\sqrt{-1}$ exclusively.

30.1 Representing polynomials

The coefficient and point-value representations of polynomials are in a sense equivalent: a polynomial in point-value form has a unique counterpart in coefficient form. This section introduces the two representations and shows how to combine them in order to multiply two degree-bound n polynomials in $\Theta(n \lg n)$ time.

Coefficient representation

A **coefficient representation** of a polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$ of degree-bound n is a vector of coefficients $a = (a_0, a_1, \dots, a_{n-1})$. Matrix equations in this chapter generally treat vectors as column vectors.

The coefficient representation is convenient for certain operations on polynomials. For example, the operation of **evaluating** the polynomial $A(x)$ at a given point x_0 consists of computing the value of $A(x_0)$. To evaluate a polynomial in $\Theta(n)$ time, use **Horner's rule**:

$$A(x_0) = a_0 + x_0 \left(a_1 + x_0 \left(a_2 + \cdots + x_0 (a_{n-2} + x_0 (a_{n-1})) \cdots \right) \right).$$

Similarly, adding two polynomials represented by the coefficient vectors $a = (a_0, a_1, \dots, a_{n-1})$ and $b = (b_0, b_1, \dots, b_{n-1})$ takes $\Theta(n)$ time: just produce the coefficient vector $c = (c_0, c_1, \dots, c_{n-1})$, where $c_j = a_j + b_j$ for $j = 0, 1, \dots, n-1$.

Now, consider multiplying two degree-bound n polynomials $A(x)$ and $B(x)$ represented in coefficient form. The method described by equations (30.1) and (30.2) takes $\Theta(n^2)$ time, since it multiplies each coefficient in the vector a by each co-

efficient in the vector b . The operation of multiplying polynomials in coefficient form seems to be considerably more difficult than that of evaluating a polynomial or adding two polynomials. The resulting coefficient vector c , given by equation (30.2), is also called the *convolution* of the input vectors a and b , denoted $c = a \otimes b$. Since multiplying polynomials and computing convolutions are fundamental computational problems of considerable practical importance, this chapter concentrates on efficient algorithms for them.

Point-value representation

A *point-value representation* of a polynomial $A(x)$ of degree-bound n is a set of n *point-value pairs*

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$$

such that all of the x_k are distinct and

$$y_k = A(x_k) \tag{30.3}$$

for $k = 0, 1, \dots, n-1$. A polynomial has many different point-value representations, since any set of n distinct points $x_0, x_1, \dots, x_{n-1}$ can serve as a basis for the representation.

Computing a point-value representation for a polynomial given in coefficient form is in principle straightforward, since all you have to do is select n distinct points $x_0, x_1, \dots, x_{n-1}$ and then evaluate $A(x_k)$ for $k = 0, 1, \dots, n-1$. With Horner's method, evaluating a polynomial at n points takes $\Theta(n^2)$ time. We'll see later that if you choose the points x_k cleverly, you can accelerate this computation to run in $\Theta(n \lg n)$ time.

The inverse of evaluation—determining the coefficient form of a polynomial from a point-value representation—is *interpolation*. The following theorem shows that interpolation is well defined when the desired interpolating polynomial must have a degree-bound equal to the given number of point-value pairs.

Theorem 30.1 (Uniqueness of an interpolating polynomial)

For any set $\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$ of n point-value pairs such that all the x_k values are distinct, there is a unique polynomial $A(x)$ of degree-bound n such that $y_k = A(x_k)$ for $k = 0, 1, \dots, n-1$.

Proof The proof relies on the existence of the inverse of a certain matrix. Equation (30.3) is equivalent to the matrix equation

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}. \quad (30.4)$$

The matrix on the left is denoted $V(x_0, x_1, \dots, x_{n-1})$ and is known as a **Vandermonde matrix**. By Problem D-1 on page 1223, this matrix has determinant

$$\prod_{0 \leq j < k \leq n-1} (x_k - x_j),$$

and therefore, by Theorem D.5 on page 1221, it is invertible (that is, nonsingular) if the x_k are distinct. To solve for the coefficients a_j uniquely given the point-value representation, use the inverse of the Vandermonde matrix:

$$a = V(x_0, x_1, \dots, x_{n-1})^{-1} y. \quad \blacksquare$$

The proof of Theorem 30.1 describes an algorithm for interpolation based on solving the set (30.4) of linear equations. Section 28.1 shows how to solve these equations in $O(n^3)$ time.

A faster algorithm for n -point interpolation is based on **Lagrange's formula**:

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}. \quad (30.5)$$

You might want to verify that the right-hand side of equation (30.5) is a polynomial of degree-bound n that satisfies $A(x_k) = y_k$ for all k . Exercise 30.1-5 asks you how to compute the coefficients of A using Lagrange's formula in $\Theta(n^2)$ time.

Thus, n -point evaluation and interpolation are well-defined inverse operations that transform between the coefficient representation of a polynomial and a point-value representation.¹ The algorithms described above for these problems take $\Theta(n^2)$ time.

The point-value representation is quite convenient for many operations on polynomials. For addition, if $C(x) = A(x) + B(x)$, then $C(x_k) = A(x_k) + B(x_k)$ for any point x_k . More precisely, given point-value representations for A ,

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\},$$

¹ Interpolation is a notoriously tricky problem from the point of view of numerical stability. Although the approaches described here are mathematically correct, small differences in the inputs or round-off errors during computation can cause large differences in the result.

and for B ,

$$\{(x_0, y'_0), (x_1, y'_1), \dots, (x_{n-1}, y'_{n-1})\} ,$$

where A and B are evaluated at the *same* n points, then a point-value representation for C is

$$\{(x_0, y_0 + y'_0), (x_1, y_1 + y'_1), \dots, (x_{n-1}, y_{n-1} + y'_{n-1})\} .$$

Thus the time to add two polynomials of degree-bound n in point-value form is $\Theta(n)$.

Similarly, the point-value representation is convenient for multiplying polynomials. If $C(x) = A(x)B(x)$, then $C(x_k) = A(x_k)B(x_k)$ for any point x_k , and to obtain a point-value representation for C , just pointwise multiply a point-value representation for A by a point-value representation for B . Polynomial multiplication differs from polynomial addition in one key aspect, however: $\text{degree}(C) = \text{degree}(A) + \text{degree}(B)$, so that if A and B have degree-bound n , then C has degree-bound $2n$. A standard point-value representation for A and B consists of n point-value pairs for each polynomial. Multiplying these together gives n point-value pairs, but $2n$ pairs are necessary to interpolate a unique polynomial C of degree-bound $2n$. (See Exercise 30.1-4.) Instead, begin with “extended” point-value representations for A and for B consisting of $2n$ point-value pairs each. Given an extended point-value representation for A ,

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{2n-1}, y_{2n-1})\} ,$$

and a corresponding extended point-value representation for B ,

$$\{(x_0, y'_0), (x_1, y'_1), \dots, (x_{2n-1}, y'_{2n-1})\} ,$$

then a point-value representation for C is

$$\{(x_0, y_0 y'_0), (x_1, y_1 y'_1), \dots, (x_{2n-1}, y_{2n-1} y'_{2n-1})\} .$$

Given two input polynomials in extended point-value form, multiplying them to obtain the point-value form of the result takes just $\Theta(n)$ time, much less than the $\Theta(n^2)$ time required to multiply polynomials in coefficient form.

Finally, let's consider how to evaluate a polynomial given in point-value form at a new point. For this problem, the simplest approach known is to first convert the polynomial to coefficient form and then evaluate it at the new point.

Fast multiplication of polynomials in coefficient form

Can the linear-time multiplication method for polynomials in point-value form expedite polynomial multiplication in coefficient form? The answer hinges on

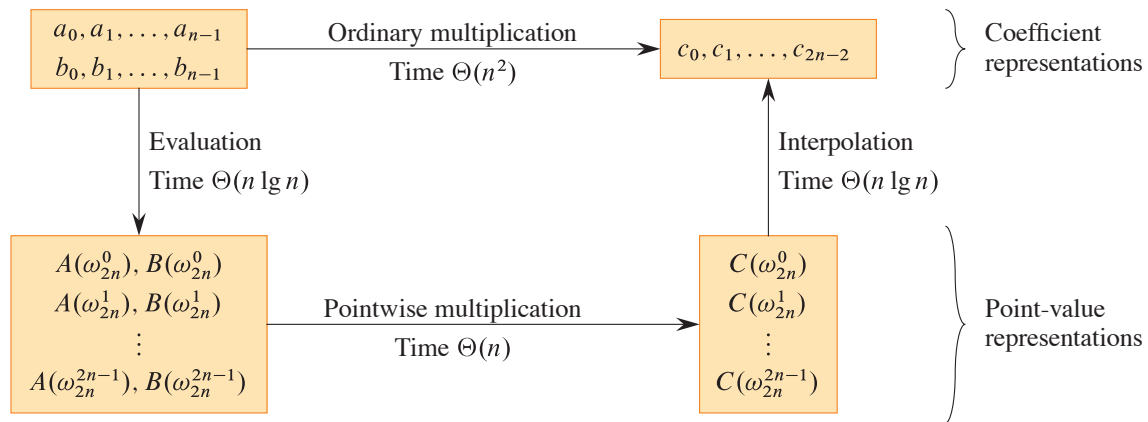


Figure 30.1 A graphical outline of an efficient polynomial-multiplication process. Representations on the top are in coefficient form, and those on the bottom are in point-value form. The arrows from left to right correspond to the multiplication operation. The ω_{2n} terms are complex $(2n)$ th roots of unity.

whether it is possible convert a polynomial quickly from coefficient form to point-value form (evaluate) and vice versa (interpolate).

Any points can serve as evaluation points, but certain evaluation points allow conversion between representations in only $\Theta(n \lg n)$ time. As we'll see in Section 30.2, if “complex roots of unity” are the evaluation points, then the discrete Fourier transform (or DFT) evaluates and the inverse DFT interpolates. Section 30.2 shows how the FFT accomplishes the DFT and inverse DFT operations in $\Theta(n \lg n)$ time.

Figure 30.1 shows this strategy graphically. One minor detail concerns degree-bounds. The product of two polynomials of degree-bound n is a polynomial of degree-bound $2n$. Before evaluating the input polynomials A and B , therefore, first double their degree-bounds to $2n$ by adding n high-order coefficients of 0. Because the vectors have $2n$ elements, use “complex $(2n)$ th roots of unity,” which are denoted by the ω_{2n} terms in Figure 30.1.

The following procedure takes advantage of the FFT to multiply two polynomials $A(x)$ and $B(x)$ of degree-bound n in $\Theta(n \lg n)$ -time, where the input and output representations are in coefficient form. The procedure assumes that n is an exact power of 2, so if it isn't, just add high-order zero coefficients.

1. **Double degree-bound:** Create coefficient representations of $A(x)$ and $B(x)$ as degree-bound $2n$ polynomials by adding n high-order zero coefficients to each.

2. **Evaluate:** Compute point-value representations of $A(x)$ and $B(x)$ of length $2n$ by applying the FFT of order $2n$ on each polynomial. These representations contain the values of the two polynomials at the $(2n)$ th roots of unity.
3. **Pointwise multiply:** Compute a point-value representation for the polynomial $C(x) = A(x)B(x)$ by multiplying these values together pointwise. This representation contains the value of $C(x)$ at each $(2n)$ th root of unity.
4. **Interpolate:** Create the coefficient representation of the polynomial $C(x)$ by applying the FFT on $2n$ point-value pairs to compute the inverse DFT.

Steps (1) and (3) take $\Theta(n)$ time, and steps (2) and (4) take $\Theta(n \lg n)$ time. Thus, once we show how to use the FFT, we will have proven the following.

Theorem 30.2

Two polynomials of degree-bound n with both the input and output representations in coefficient form can be multiplied in $\Theta(n \lg n)$ time. ■

Exercises

30.1-1

Multiply the polynomials $A(x) = 7x^3 - x^2 + x - 10$ and $B(x) = 8x^3 - 6x + 3$ using equations (30.1) and (30.2).

30.1-2

Another way to evaluate a polynomial $A(x)$ of degree-bound n at a given point x_0 is to divide $A(x)$ by the polynomial $(x - x_0)$, obtaining a quotient polynomial $q(x)$ of degree-bound $n - 1$ and a remainder r , such that

$$A(x) = q(x)(x - x_0) + r.$$

Then we have $A(x_0) = r$. Show how to compute the remainder r and the coefficients of $q(x)$ from x_0 and the coefficients of A in $\Theta(n)$ time.

30.1-3

Given a polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$, define $A^{\text{rev}}(x) = \sum_{j=0}^{n-1} a_{n-1-j} x^j$. Show how to derive a point-value representation for $A^{\text{rev}}(x)$ from a point-value representation for $A(x)$, assuming that none of the points is 0.

30.1-4

Prove that n distinct point-value pairs are necessary to uniquely specify a polynomial of degree-bound n , that is, if fewer than n distinct point-value pairs are given, they fail to specify a unique polynomial of degree-bound n . (*Hint:* Using Theorem 30.1, what can you say about a set of $n - 1$ point-value pairs to which you add one more arbitrarily chosen point-value pair?)

30.1-5

Show how to use equation (30.5) to interpolate in $\Theta(n^2)$ time. (*Hint*: First compute the coefficient representation of the polynomial $\prod_j (x - x_j)$ and then divide by $(x - x_k)$ as necessary for the numerator of each term (see Exercise 30.1-2). You can compute each of the n denominators in $O(n)$ time.)

30.1-6

Explain what is wrong with the “obvious” approach to polynomial division using a point-value representation: dividing the corresponding y values. Discuss separately the case in which the division comes out exactly and the case in which it doesn’t.

30.1-7

Consider two sets A and B , each having n integers in the range from 0 to $10n$. The **Cartesian sum** of A and B is defined by

$$C = \{x + y : x \in A \text{ and } y \in B\} .$$

The integers in C lie in the range from 0 to $20n$. Show how, in $O(n \lg n)$ time, to find the elements of C and the number of times each element of C is realized as a sum of elements in A and B . (*Hint*: Represent A and B as polynomials of degree at most $10n$.)

30.2 The DFT and FFT

In Section 30.1, we claimed that by computing the DFT and its inverse by using the FFT, it is possible to evaluate and interpolate a degree n polynomial at the complex roots of unity in $\Theta(n \lg n)$ time. This section defines complex roots of unity, studies their properties, defines the DFT, and then shows how the FFT computes the DFT and its inverse in $\Theta(n \lg n)$ time.

Complex roots of unity

A **complex n th root of unity** is a complex number ω such that

$$\omega^n = 1 .$$

There are exactly n complex n th roots of unity: $e^{2\pi i k/n}$ for $k = 0, 1, \dots, n-1$. To interpret this formula, use the definition of the exponential of a complex number:

$$e^{iu} = \cos(u) + i \sin(u) .$$

Figure 30.2 shows that the n complex roots of unity are equally spaced around the circle of unit radius centered at the origin of the complex plane. The value

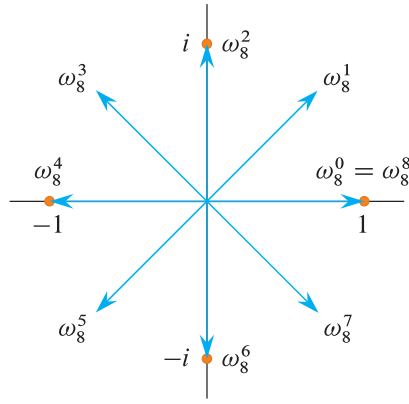


Figure 30.2 The values of $\omega_8^0, \omega_8^1, \dots, \omega_8^7$ in the complex plane, where $\omega_8 = e^{2\pi i/8}$ is the principal 8th root of unity.

$$\omega_n = e^{2\pi i/n} \quad (30.6)$$

is the *principal n th root of unity*.² All other complex n th roots of unity are powers of ω_n .

The n complex n th roots of unity,

$$\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1},$$

form a group under multiplication (see Section 31.3). This group has the same structure as the additive group $(\mathbb{Z}_n, +)$ modulo n , since $\omega_n^n = \omega_n^0 = 1$ implies that $\omega_n^j \omega_n^k = \omega_n^{j+k} = \omega_n^{(j+k) \bmod n}$. Similarly, $\omega_n^{-1} = \omega_n^{n-1}$. The following lemmas furnish some essential properties of the complex n th roots of unity.

Lemma 30.3 (Cancellation lemma)

For any integers $n > 0$, $k \geq 0$, and $d > 0$,

$$\omega_{dn}^{dk} = \omega_n^k. \quad (30.7)$$

Proof The lemma follows directly from equation (30.6), since

$$\begin{aligned} \omega_{dn}^{dk} &= (e^{2\pi i/dn})^{dk} \\ &= (e^{2\pi i/n})^k \\ &= \omega_n^k. \end{aligned}$$

■

² Many other authors define ω_n differently: $\omega_n = e^{-2\pi i/n}$. This alternative definition tends to be used for signal-processing applications. The underlying mathematics is substantially the same with either definition of ω_n .

Corollary 30.4

For any even integer $n > 0$,

$$\omega_n^{n/2} = \omega_2 = -1 .$$

Proof The proof is left as Exercise 30.2-1. ■

Lemma 30.5 (Halving lemma)

If $n > 0$ is even, then the squares of the n complex n th roots of unity are the $n/2$ complex $(n/2)$ th roots of unity.

Proof By the cancellation lemma, $(\omega_n^k)^2 = \omega_{n/2}^k$ for any nonnegative integer k . Squaring all of the complex n th roots of unity produces each $(n/2)$ th root of unity exactly twice, since

$$\begin{aligned} (\omega_n^{k+n/2})^2 &= \omega_n^{2k+n} \\ &= \omega_n^{2k} \omega_n^n \\ &= \omega_n^{2k} \\ &= (\omega_n^k)^2 . \end{aligned}$$

Thus ω_n^k and $\omega_n^{k+n/2}$ have the same square. We could also have used Corollary 30.4 to prove this property, since $\omega_n^{n/2} = -1$ implies $\omega_n^{k+n/2} = \omega_n^k \omega_n^{n/2} = -\omega_n^k$, and thus $(\omega_n^{k+n/2})^2 = (-\omega_n^k)^2 = (\omega_n^k)^2$. ■

As we'll see, the halving lemma is essential to the divide-and-conquer approach for converting between coefficient and point-value representations of polynomials, since it guarantees that the recursive subproblems are only half as large.

Lemma 30.6 (Summation lemma)

For any integer $n \geq 1$ and nonzero integer k not divisible by n ,

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = 0 .$$

Proof Equation (A.6) on page 1142 applies to complex values as well as to reals, giving

$$\begin{aligned}
\sum_{j=0}^{n-1} (\omega_n^k)^j &= \frac{(\omega_n^k)^n - 1}{\omega_n^k - 1} \\
&= \frac{(\omega_n^n)^k - 1}{\omega_n^k - 1} \\
&= \frac{(1)^k - 1}{\omega_n^k - 1} \\
&= 0.
\end{aligned}$$

To see that the denominator is not 0, note that $\omega_n^k = 1$ only when k is divisible by n , which the lemma statement prohibits. ■

The DFT

Recall the goal of evaluating a polynomial

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

of degree-bound n at $\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}$ (that is, at the n complex n th roots of unity).³ The polynomial A is given in coefficient form: $a = (a_0, a_1, \dots, a_{n-1})$. Let us define the results y_k , for $k = 0, 1, \dots, n-1$, by

$$\begin{aligned}
y_k &= A(\omega_n^k) \\
&= \sum_{j=0}^{n-1} a_j \omega_n^{kj}.
\end{aligned} \tag{30.8}$$

The vector $y = (y_0, y_1, \dots, y_{n-1})$ is the **discrete Fourier transform (DFT)** of the coefficient vector $a = (a_0, a_1, \dots, a_{n-1})$. We also write $y = \text{DFT}_n(a)$.

The FFT

The **fast Fourier transform (FFT)** takes advantage of the special properties of the complex roots of unity to compute $\text{DFT}_n(a)$ in $\Theta(n \lg n)$ time, as opposed to the $\Theta(n^2)$ time of the straightforward method. Assume throughout that n is an exact power of 2. Although strategies for dealing with sizes that are not exact powers of 2 are known, they are beyond the scope of this book.

³ The length n is actually what Section 30.1 referred to as $2n$, since the degree-bound of the given polynomials doubles prior to evaluation. In the context of polynomial multiplication, therefore, we are actually working with complex $(2n)$ th roots of unity.